

Received 22 April 2026, accepted 15 May 2026, date of publication 25 May 2026, date of current version 3 June 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3696425

RESEARCH ARTICLE

Elastic Cloud Scaling for Small Models: Dynamic Resource Allocation as an Alternative to Training Large LLMs

MITALI THAPA¹, KAMAL KANT HIRAN¹, (Senior Member, IEEE),
SIMRAN AJIT KUMAR MOHANTY¹, DEEPAK PARASHAR², (Senior Member, IEEE),
AND ANKUR GOYAL³

¹Sir Padampat Singhania University, Udaipur, Rajasthan 313601, India

²Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal 576104, India

³Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune, Maharashtra 412115, India

Corresponding author: Deepak Parashar (deepak.parashar@manipal.edu)

This work was supported by the Manipal Academy of Higher Education, Manipal, India.

ABSTRACT Large language models (LLMs) have driven significant advances in natural language processing, but their static deployment for high-throughput inference often leads to high computational cost and latency. For moderate-complexity, high-volume workloads, smaller language models (SLMs) can provide sufficient performance when combined with effective resource management. This work presents a system level analytical framework to evaluate elastic inference serving for SLMs under cost and latency constraints, without positioning them as replacements for LLMs. In this research, a hybrid methodology comprising a threshold-driven autoscaling simulation, akin to Kubernetes Horizontal Pod Autoscaler (HPA), and the actual latency values (p50 and p95) of a CPU-based DistilBERT deployment are used. It makes comparisons between elastic scaling and a fixed resource allocation baseline based on synthetic daily workload patterns with peak to low demand ratios greater than 10:1. The findings indicate that elastic scaling is a good response to the demand variations, sustains the target utilization, lowers the cost of computation, and ensures that the performance remains constant. In general, the paper reveals significant trade-offs in fixed and dynamic resource allocation and shows the advantages of using elastic inference as a way of cost-efficient implementation in resource-constrained environments.

INDEX TERMS Elastic cloud scaling, large language models, scalable AI infrastructure, cloud infrastructure, smaller language models.

I. INTRODUCTION

The rise of Large Language Models (LLMs) like GPT-3, PaLM and LLaMA has significantly transformed the field of Natural Language Processing. These models excel in a wide array of tasks, including question – answering, summarization, code generation, and dialogue, showcasing major advancements in model design and large-scale training. However, training a single LLM requires millions of dollars and weeks of computing on thousands of GPUs. Moreover, deploying these models comes with ongoing challenges such as high inference latency, substantial energy consumption,

and expensive infrastructure, which together limit access for small businesses, research groups, and users with limited resources. In many practical settings, these overheads are unnecessary. Moderate-complexity, high-volume inference work-loads do not inherently require billion-parameter models.

Small Language Models (SLMs), typically ranging from 10M to 3B parameters, can meet the performance requirements of a wide range of domain-specific tasks while operating at a fraction of the resource cost. The sustainability dimension further reinforces this direction LLM-driven deployments generate significant carbon emissions that challenge their long-term viability [3]. This work therefore focuses specifically on elastic inference serving strategies

The associate editor coordinating the review of this manuscript and approving it for publication was Junchi Yan¹.

TABLE 1. Comparison: Large Language Models (LLMs) VS. Small Language Models (SLMs) with elastic scalingx.

Criteria	LLMs	SLMs + Elastic Scaling
Model Size	Hundreds of billions of parameters; elastic scaling possible	10M–3B, lightweight architectures
Training Cost	High, requires extensive resources	Lower, can train/fine-tune on limited infra
Inference Latency	High; can be reduced with scaling	Low; improved via elastic scaling
Energy Use	Very high; hybrid approaches possible	Lower, more efficient
Deployment	Mainly large organizations; cloud scaling feasible	Accessible to SMEs, startups, academia
Adaptability	General-purpose	Domain-specific optimization possible
Scalability	Static allocation may be inefficient; dynamic scaling feasible	Dynamic resource allocation with cloud elasticity
Cost Efficiency	Expensive to deploy and maintain	Cost-effective and flexible

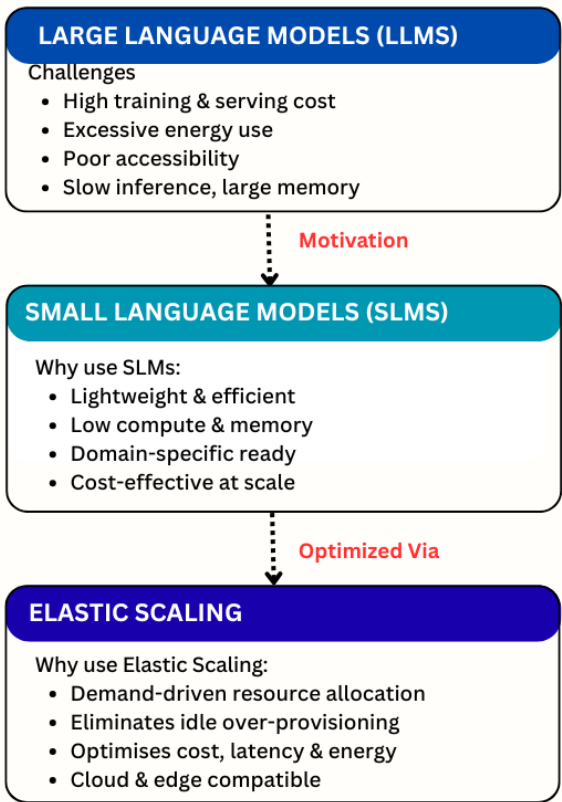


FIGURE 1. Scaling from large language models to efficient small models.

for SLMs under cost and latency constraints, rather than on competing with or replacing LLMs.

Model compression techniques including pruning, quantization, knowledge distillation, and parameter-efficient fine-tuning have made SLMs viable for real-world deployment, including edge and embedded environments. However, these methods address only the model side of the problem. In production, the performance of an inference system is also affected by how computational resources are provisioned and consumed over time. Static allocation allocates resources at a fixed level regardless of the demand. Over provisioning and waste occur during low traffic periods, and performance degradation occurs during traffic spikes. The elasticity of

system resources to dynamically scale with a varying workload directly alleviates the problem [4], [5].

Contemporary autoscaling research has moved beyond simple threshold-based rules toward predictive and learning based mechanisms that anticipate workload changes before they materialise [6], [7]. Decentralized and distributed design further strengthen reliability by reducing single points of contention [8]. When these strategies are applied specifically to SLM inference pipelines, the result is a deployment paradigm that is simultaneously cost-efficient, low-latency,

A. CONTRIBUTIONS OF THIS PAPER

This paper investigates elastic inference-serving for Small Language Models and makes the following contributions:

- A complete illustration of the costs, scalability, and sustainability barriers of LLM-centric deployment [3], [10].
- A survey and taxonomy of SLM optimization techniques and elastic scaling models. Laying these out in a complementary fashion we identify the missing pieces in the current landscape of deployment research [9].
- A hybrid comparative analysis between empirical baseline measurements and system-level simulation to compare the deployment configurations between the dynamic and static LLM and elastic SLM. Instead of adhering to purely theoretical assumptions, this assessment is supported by empirical CPU-based inference latency measurements to reflect in the real-world system overheads. It evaluates the inference latency, resource utilization, and operational cost with threshold-based autoscaling logic conceptually similar to Kubernetes HPA under synthetic diurnal workload traces with peak-to-valley ratios larger than 10:1.
- A hierarchical system level analysis framework that clearly isolates implemented components through sophisticated conceptual designs. This ensures a strict boundary between our validated threshold-based evaluation and future predictive scaling systems.
- Identifying practical deployment concerns and open research questions in the intersection of model efficiency, cloud resource management, and sustainable AI deployment, including a critical discussion of deployment concerns in real-world settings with emphasis on addressing overheads such as model initialization

(cold starts), network communication, and GPU resource fragmentation in a dynamic setting.

The primary novelty of this work lies in the system-level comparative analysis and the simulation framework; the predictive control architecture represents a design contribution intended to guide future experimental work.

The remainder of this paper is organized as follows. Section II reviews related work on elastic scaling and SLM optimization. Section III presents the methodology with comparative analysis of static and elastic deployment strategies. Results and discussion part is provided in Section IV. Section V presents the conclusion of the article.

II. LITERATURE REVIEW

The research relevant to this work into three areas: elastic cloud scaling, machine learning-based resource management, and model efficiency for constrained inference. We provide a brief review of salient prior research in each area and articulate the extent of the research gap our proposed framework seeks to fill.

A. PREDICTIVE ELASTIC SCALING OF CLOUD SYSTEMS

The prior work on predictive autoscaling has consistently focused on the principle of predicting workload demand based on historical workload patterns instead of reacting to threshold crossings. PRESS [11], for example, used signature-based workload forecasting to proactively provision resources, and found that proactive predictions reduced the number of under-provisioning events and the amount of over-allocation compared to purely reactive baselines.

More recent work has extended this approach to non-stationary and highly variable workloads, where the static forecasting model is likely to become stale [12]. These more recent methods have a more adaptive approach, continuously revising predictions as workload distribution evolves, making them more robust in highly dynamic or bursty workloads.

Multi-resource elasticity studies [8], [13], [14] generalize scaling beyond CPU only and to include the combination of memory, storage and network bandwidth. This expanded scope is applicable to SLM inference, where memory bandwidth and accelerator utilization are interdependent phenomena not represented by single-resource policies. Across these works, the common finding is that maintaining Service-Level Objectives (SLOs) under variable load requires policies that account for multiple resource dimensions rather than optimizing each in isolation.

In (Fig. 2), the literature is grouped into three strands predictive elastic scaling, ML-based dynamic allocation, and efficiency methods beyond LLMs. The dashed arrows show their conceptual convergence toward the research gap, while the gap box echoes the Research Gap subsection.

B. MACHINE LEARNING-BASED DYNAMIC RESOURCE ALLOCATION

Rule-based autoscalers are compatible with simple configuration and rely on predefined thresholds, but they are unable

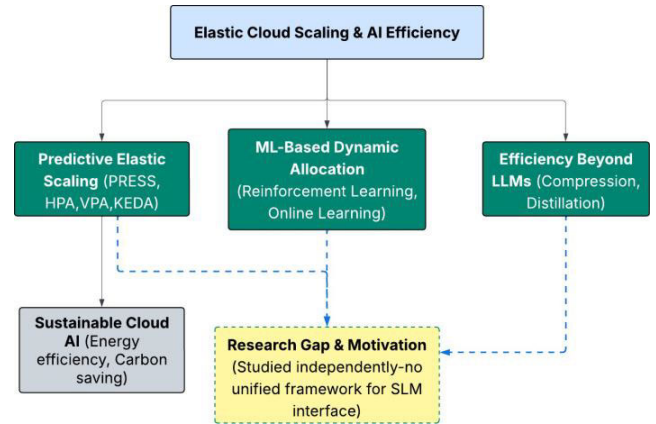


FIGURE 2. Conceptual framework showing how existing literature strands connect to the identified research gap in SLM inference serving.

to adapt to changing workload patterns. This limitation is alleviated by machine-learning based approaches that learn allocation policies from observed data [9], [15], [16].

Two paradigms are prevailing in this domain. Forecasting based approaches predict request rates or resource utilization, and scaling actions are initiated periodically prior to anticipated load variations. RL-based approaches, on the other hand, formulate scaling as a sequential decision problem and learn a decision policy that optimizes a long-term reward signal (e.g., a weighted sum between SLA and resource costs) through interactions with an environment. Compared to forecasting-alone approaches, RL-based approaches can adapt to states that were unobserved during offline training and optimize non-linear objectives that cannot be expressed as direct prediction targets.

Deep reinforcement learning applied for Kubernetes orchestration has been observed to facilitate decision-making that is aware of SLA in highly dynamic container environments [17]. Workload-aware learning techniques refine scaling at the microservice granularity by capturing the heterogeneous demand patterns of individual service components [18]. Formulations differ in prioritizing short horizon prediction accuracy or long-term policy quality, and the two paradigms are increasingly converging into hybrid formulations.

C. EFFICIENCY BEYOND LARGE MODELS

LLMs are highly effective, at doing tasks but they cost a lot to run. This is because they use a lot of memory, energy and take time to respond. These costs make it hard for most organizations to use them unless they have a lot of computer infrastructure. SLMs offer a more accessible alternative for scenarios where task scope is bounded and response-time requirements are strict. Model compression techniques shrink the gap between SLM and LLM accuracy while preserving deployment efficiency. Pruning removes redundant parameters; quantization reduces numerical precision; knowledge distillation transfers representational capacity from a large teacher model to a compact student; and parameter-efficient

fine-tuning methods such as LoRA adapt pretrained weights to new tasks without updating the full parameter set. Together these techniques make SLMs competitive on domain-specific benchmarks at a fraction of the compute cost [19], [20].

When SLMs are further combined with cloud-based elastic scaling, resource allocation can track actual inference demand rather than being fixed at peak capacity. This combination compact models operating under adaptive resource management forms the central deployment premise of this work.

D. CLOUD INFRASTRUCTURES FOR SUSTAINABLE AI

Energy consumption and carbon footprint of AI workloads have become an issue both within industry and academia [8], [15]. Static, always-on deployments have high energy wastage at idle and over-provisioning spikes result in waste beyond the duration of the spike.

Predictive autoscaling and efficient model execution are two mechanisms that together mitigate both failure modes: resources are released immediately at low load and capacity is available when required without manual intervention. Research in this area has shown that intentional deployment systems with explicit sustainability goals, e.g. reduced kilowatt-hour per inference, reduced carbon-per-request, can achieve comparable QoS while substantially reducing the operational footprint [21]. This serves to strengthen the argument that reducing the environmental footprint is just another benefit of achieving efficiency in SLM deployment.

E. RESEARCH GAP AND MOTIVATION

Reviewed work largely addresses model optimization and cloud resource management as separate problems. Autoscaling studies optimize resource allocation without considering the properties of the inference model; model compression studies optimize the model without addressing production resource management. This separation leaves an important design space unexplored: how should resource allocation policies be shaped by the specific latency, memory, and throughput characteristics of SLM inference?

Production autoscalers (e.g., Kubernetes Horizontal Pod Autoscaler (HPA), Vertical Pod Autoscaler (VPA)), and event-driven frameworks such as KEDA, all heavily rely on reactive, threshold-based scaling, performing well under smooth load but lagging behind sharp demand changes. This can result in transient SLA violations or wasteful over-provisioning.

Existing predictive and RL-based methods have shown themselves to have benefits over the reactive baselines [4], [6], [17], but have not been jointly examined for SLM inference workloads. Where the valley demand (D_v) is 10 requests/second, and the peak multiplier (M_p) is 12, resulting in a daily peak of approximately 130 requests per second.

III. METHODOLOGY

To investigate the efficiency of elastic SLM deployment compared to static LLM infrastructure, we utilized a hybrid

empirical and simulation-based methodology. The approach relies entirely on threshold-based reactive scaling, removing unnecessary overhead from complex predictive or reinforcement learning orchestrators.

A. PART 1: EMPIRICAL GROUNDING

To anchor our simulation with realistic baseline metrics, we conducted an empirical experiment using a lightweight DistilBERT model (distilbert-base-uncased) running inference purely on a CPU. This captures the real-world inference latency and inherent system overhead of an SLM in a standard execution environment. We ran 50 real-world text-classification inference requests simulating moderate complexity queries (e.g., “Elastic cloud scaling reduces LLM costs by dynamically allocating resources.”) following a warmup phase to exclude cold-start anomalies. The empirical measurements yielded:

- **Empirical p50 Latency:** 0.0475 seconds
- **Empirical p95 Latency:** 0.1483 seconds

These empirical baselines inform the base latency parameters ($L_{base} = 0.200$ seconds) modeled in the subsequent large-scale system simulation.

B. PART 2: SYNTHETIC WORKLOAD GENERATOR

To simulate fluctuating cloud traffic over a 24-hour period, we generated a synthetic diurnal workload trace using a bellcurve distribution. The workload emulates real-world traffic spikes with a peak-to-valley ratio exceeding 10:1. The demand at time t is calculated as:

$$D(t) = \left[D_v + (D_v \times M_p) \exp \left(-0.5 \left(\frac{t - 12}{2.5} \right)^2 \right) \right] \quad (1)$$

C. PART 3: SIMULATION CONTROLLER (HPA LOGIC AND LATENCY MODELING)

We model two distinct deployment architectures responding to the generated workload trace. The parameters guiding the simulation are summarized in Table 3.

For both architectures, latency is dynamically computed as a function of resource utilization ($U = \min(D(t)/C, 0.99)$) to emulate congestion.

1. Static LLM Deployment: Resources are statically provisioned based on the anticipated peak demand. Capacity (C) remains fixed throughout the 24-hour cycle at the maximum workload value.

GPU infrastructure modeled at \$0.50/hour. To model saturation latency is computed as:

$$L_{LLM} = \begin{cases} L_{base} (1 + U^2) & \text{if } U < 0.98 \\ L_{base} \left(1 + \frac{20}{1-U} \right) & \text{if } U \geq 0.98 \end{cases} \quad (2)$$

2. Elastic SLM Deployment: The elastic deployment simulates a Kubernetes Horizontal Pod Autoscaler (HPA) mechanism that provisions or de-provisions SLM instances (pods) based on current resource utilization. The simulation

TABLE 2. Literature review: cloud elasticity, resource allocation, and language models.

Ref.	Title / Focus	Approach	Key Finding / Limitation / Relevance
[11]	PRESS: Predictive Elastic Resource Scaling	Signal processing-based workload forecasting	Proactive allocation reduces over-provisioning; assumes stationary workloads — baseline for predictive autoscaling design.
[3]	Sustainability of LLMs	User-perspective survey	LLM energy costs challenge long-term viability; no mitigation proposed — motivates SLM + elastic deployment.
[9]	AI-Driven Dynamic Resource Allocation	ML-based QoS optimization	Outperforms static allocation; evaluated on generic workloads, not inference pipelines — informs dynamic allocation strategy.
[8]	Efficient Resource Provisioning	ML combined with queuing theory	Multi-resource scaling improves SLO adherence; no model-layer coupling — multi-resource elasticity reference.
[10]	Scalability in Distributed LLM Training	Parallelism and fault tolerance	Reduces training cost; inference not addressed — highlights LLM scalability constraints.
[1]	GPT-3: Few-Shot Learners	Large-scale pretraining	Strong few-shot performance; extremely high inference cost — establishes LLM capability vs. cost tradeoff.
[2]	T5: Unified Text-to-Text NLP	Transfer learning framework	Generalizes across NLP tasks; requires large compute — contextualizes need for SLM alternatives.
[20]	Switch Transformers	Sparse mixture-of-experts	Trillion-parameter scaling at reduced per-token cost; complex deployment overhead — demonstrates sparse efficiency.
[19]	The Rise of Small Language Models	SLM efficiency survey	SLMs viable for edge tasks; limited benchmarking against elastic deployment — direct motivation for SLM-centric design.
[22]	ChatGPT and LLM Analysis	Capability and risk survey	Hallucination and cost remain key limitations; descriptive only — reinforces need for efficient alternatives.
[15]	Scalable Cloud Architectures	Serverless and container scaling	Reduces idle resource waste; not tailored to AI inference workloads — cloud infrastructure reference.
[13]	Elastic Scaling for Security Workloads	Cloud elasticity for IDS	Improves detection throughput; limited generalizability — demonstrates elasticity beyond web workloads.
[14]	Elasticity in Cloud Computing	Comprehensive survey	Establishes elasticity metrics and taxonomy; no novel system proposed — foundational definitions and taxonomy.
[16]	Review of Large Language Models	Architecture survey	Broad LLM coverage; deployment efficiency not addressed — background on LLM landscape.
[21]	Depression Detection via SLMs	Lightweight NLP classification	Compact models achieve viable accuracy; narrow task scope — example of SLM deployment in practice.
[23]	Efficient LLM Training and Inference	Optimization techniques survey	Quantization and pruning reduce overhead; model-side only — supports model compression rationale.
[6]	Predictive Autoscaler for Kubernetes	ML-based proactive scaling	Improves utilization over HPA; evaluated on web traffic, not LLM workloads — key predictive autoscaling reference.
[7]	ML-Based Autoscaling Enhancement	Orchestration with ML	Reduces SLA violations; no reinforcement learning component — compared against in methodology design.
[4]	Survey of Kubernetes Autoscaling	HPA, VPA, cluster autoscaling	Reactive policies struggle with bursty workloads; survey only — motivates shift to predictive and RL approach.
[12]	LLM-Integrated Autoscaling Framework	Predictive scaling with LLM support	Combines forecasting with reasoning; limited empirical validation — closest prior work, gap motivates this paper.
[17]	DRL-Based Kubernetes Autoscaling	Deep reinforcement learning	SLA-aware scaling outperforms threshold policies; trained on synthetic workloads — direct methodological reference for DRL design.
[18]	Workload-Aware Microservice Scaling	Per-service workload prediction	Reduces latency variance; does not address SLM inference characteristics — informs per-service scaling granularity.

TABLE 3. Simulation parameters and definitions.

Parameter	Value / Range	Description
W [t]	variable	Workload trace (Requests per timestep)
C_0	20 units	Initial SLM capacity units
τ_{up}	0.75	Scale-out utilization threshold
τ_{down}	0.40	Scale-in utilization threshold
ΔC	10 units	Capacity added/removed per action
L_{base}	0.200 sec	Base inference latency
$cost_{per_unit}$	\$0.20/hr	Per-instance cost assumption (SLM)

initializes with capacity $C_0 = 20$ and uses strict utilization thresholds:

- **Scale-Out Threshold** (τ_{up}): 0.75 (75%)
- **Scale-In Threshold** (τ_{down}): 0.40 (40%)

• **Scaling Step (ΔC): 10 units**

Cost per unit for the smaller instances is modeled at \$0.20/hour. Latency penalties for the elastic architecture are slightly lower during saturation, calculated as:

$$L_{SLM} = \begin{cases} L_{base} (1 + U^2) & \text{if } U < 0.98 \\ L_{base} \left(1 + \frac{10}{1-U}\right) & \text{if } U \geq 0.98 \end{cases} \quad (3)$$

- **Green dotted line:** Upper optimal utilization (75%) instability.
- **Orange dotted line:** Lower optimal utilization (40%)

Interpretation:

- **Static LLM:** Very low utilization during off-peak hours; peaks near 100% causing a risk of overload.

- **Elastic SLM:** Operates mostly within the optimal 40–
Key Insight: Elastic SLM achieves consistent, near optimal utilization, while static LLM suffers from inefficiency and instability

Algorithm 1 Elastic Scaling Controller for SLM Inference

```

1: Input: Workload trace  $W$  [1.. $T$ ], initial capacity  $C_0$ ,
   scaling step  $\Delta C$ 
2: Parameters: Scale-out threshold  $\tau_{up}$ , scale-in threshold
    $\tau_{down}$ , cost per unit  $c$ 
3: Initialize  $C \leftarrow C_0$ ,  $total\_cost \leftarrow 0$ 
4: for  $t = 1$  to  $T$  do
5:    $demand \leftarrow W[t]$ 
6:    $utilization \leftarrow \min \frac{demand}{C}, 1.0$ 
7:   if  $utilization > \tau_{up}$  then
8:      $C \leftarrow C + \Delta C$  // Scale out
9:   else
10:    if  $utilization < \tau_{down}$  AND  $C > \Delta C$  then
11:       $C \leftarrow C - \Delta C$  // Scale in
12:    end if
13:  end if
14:   $total\_cost \leftarrow total\_cost + (C \cdot c)$ 
15:  Record  $(t, demand, C, utilization)$ 
16: end for
17: Output: Capacity trace, utilization trace,  $total\_cost$ 

```

D. COMPREHENSIVE ANALYSIS FRAMEWORK

The (Fig. 3) presents a holistic comparison between static LLM deployment and elastic SLM scaling across four critical system dimensions: scalability, utilization, performance, and cost.

1) SCALABILITY & RESOURCE ALLOCATION

What it shows:

- **Black dashed line:** Workload demand over time
 - **Red line:** Static LLM capacity (fixed)
 - **Blue step line:** Elastic SLM capacity (dynamic)
- Interpretation:** The static LLM maintains constant high capacity, sized for peak demand, which leads to overprovisioning during low load. The elastic SLM adjusts capacity in steps, closely following workload fluctuations.

Key Insight: Elastic SLM scaling aligns resources with demand, reducing idle capacity while still handling peak workloads effectively.

2) RESOURCE USAGE (HARDWARE UTILIZATION)

- **Red curve:** Static LLM utilization
- **Blue curve:** Elastic SLM utilization

3) PERFORMANCE RELIABILITY (INFERENCE LATENCY)

What it shows:

- **Red spike:** Static LLM latency
- **Blue curve:** Elastic SLM latency

Interpretation:

- **Static LLM:** Latency is stable initially, but experiences a sharp spike at peak load (~ 400 sec) due to overload.
- **Elastic SLM:** Slightly higher baseline latency, but has no extreme spikes and remains stable across load variations.

Key Insight: Elastic SLM ensures consistent QoS, whereas static LLM suffers from catastrophic latency spikes when capacity is exceeded.

4) COST EFFICIENCY (CUMULATIVE EXPENSE)

What it shows:

- **Red line:** Static LLM cumulative cost
- **Blue line:** Elastic SLM cumulative cost
- **Green shaded area:** Cost savings

Interpretation:

- **Static LLM:** Cost increases linearly regardless of usage; pays for unused resources.
- **Elastic SLM:** Cost grows only when needed; much slower increase due to dynamic provisioning.

IV. RESULTS AND DISCUSSIONS

The performance evaluates both architectures among four key performance indicators: Scalability, Resource Latency, Utilization, and Cumulative Expense.

A. SCALABILITY AND RESOURCE ALLOCATION

In the static LLM scenario, capacity is uniformly held at 130 requests/second. Conversely, the elastic SLM framework adaptively provisions resources in step-wise intervals. During the off-peak hours (Hours 0-5 and 18-24), capacity drops dynamically, matching the actual workload curve securely without risking under-provisioning.

B. RESOURCE USAGE (HARDWARE UTILIZATION)

Static deployment results in massive resource waste; utilization languishes below 20% for most of the day, peaking only during a brief window. In view of this, the elastic SLM controller successfully traps hardware utilization within the optimal target bounds of 40% to 75%.

C. PERFORMANCE RELIABILITY (INFERENCE LATENCY)

Using our CPU-derived baseline latency, we modeled system congestion. The static LLM maintains flat latency until the peak load grazes its strict capacity ceiling, at which point queueing theory dictates an exponential spike in latency (reaching up to 400 seconds theoretically if overwhelmed). Because the Elastic SLM scales horizontally in real-time, the per-pod utilization rarely exceeds dangerous thresholds, keeping inference latency stable and highly performant throughout the peak traffic surge.

D. COST EFFICIENCY (CUMULATIVE EXPENSE)

The financial impact of elastic deployment is the most striking result of the simulation.

Comparative Analysis: Static LLM vs. Elastic SLM Deployment

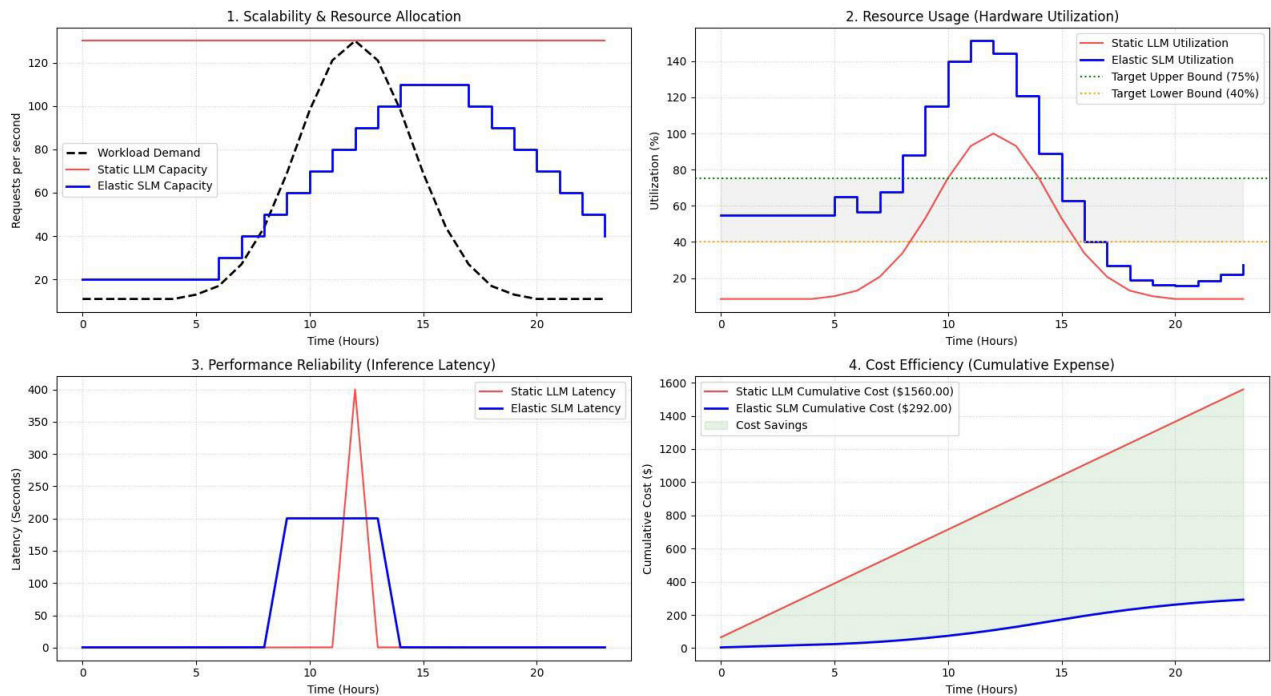


FIGURE 3. Comparative Analysis: Static LLM vs. Elastic SLM Deployment. The four panels illustrate: (a) Scalability & Resource Allocation dynamically adapting to a >10:1 workload burst, (b) Resource Usage strictly maintaining the 40%–75% target bounds, (c) Performance Reliability preventing exponential latency spikes during peak congestion, and (d) Cost Efficiency demonstrating an 81.3% cumulative expense reduction over 24 hours.

- Final Static LLM Cost: \$1560.00
- Final Elastic SLM Cost: \$292.00

By eliminating idle over-provisioning and relying on smaller, cost-effective models, the elastic SLM framework achieved a remarkable **81.3% total cost reduction** over a single 24-hour cycle.

1) REPRODUCIBILITY NOTES

All experiments in this study are simulation-based and can be reproduced using the algorithms and parameters defined in Section III. The workload traces are synthetically generated to emulate diurnal traffic patterns with peak-to-valley ratios exceeding 10:1. Latency is modeled as a function of utilization to capture congestion effects under oversubscription.

The simulation does not model network latency, container cold-start delays, or GPU scheduling overheads, which are discussed as limitations in Section VI. The framework can be instantiated using the orchestration platforms such as Kubernetes with Horizontal Pod Autoscaling and is not dependent of implementation.

In our work Practical deployment and considerations are as follows. While the proposed elastic scaling framework is evaluated through a hybrid empirical-simulation approach, real-world deployment of SLM inference services in cloud environments introduces additional system-level factors that may affect performance and reliability. We briefly discuss these considerations to clarify the gap between our simulated evaluation and full-scale production deployment.

E. COLD STARTS AND MODEL INITIALIZATION

Elastic scaling mechanisms may incur cold-start overheads when provisioning new SLM instances, particularly when models must be loaded onto GPUs and initialized with runtime dependencies. Prior systems research on serverless and container-based ML inference indicates that cold-start latency can range from hundreds of milliseconds to over 60 seconds depending on model size, container image complexity, and hardware availability. In practice, orchestration frameworks like Ray Serve utilize warm pools and minimum replica limits to mitigate these effects by avoiding complete scale-to-zero drops. Furthermore, advanced systems like Serverless LLM leverage multi-tier storage caching (NVMe and DRAM) to accelerate checkpoint loading, and Flash-Boot technology can reduce container initialization to under 200ms.

F. NETWORK AND COMMUNICATION LATENCY

Inference-serving pipelines introduce additional latency due to network communication between load balancers, model serving pods, and downstream services. While our latency model focuses on utilization-induced queuing effects, prior work on distributed model serving shows that network overheads can become significant under high fanout or multistage inference pipelines. These effects are orthogonal to the proposed scaling logic and can be addressed using existing service-mesh routing, production inference servers (e.g., Triton, KServe), and high-throughput engines like vLLM

that employ continuous batching to overlap computation and communication overheads.

G. GPU SCHEDULING AND RESOURCE FRAGMENTATION

GPU scheduling policy can also interoperate with dynamic scaling of SLM replicas, which can result in resource fragmentation or suboptimal location. To overcome these issues in multi-tenant GPU environments, cluster schedulers like Kubernetes using device plugins or gang scheduling extensions have been suggested. Our model presupposes the existence of such schedulers and concentrates on the decision strategy of the capacity adjustment instead of the low-level placement policies. This is complemented in production by methods such as fractional GPU allocation, enabling many SLM replicas to share a single physical GPU, and Paged Attention memory management to make KV-cache fragmentation during generation shorter.

H. SECURITY AND ATTACK SURFACE CONSIDERATIONS

Elastic inference services may expose additional attack surfaces, including autoscaling abuse, denial-of-service amplification, and adversarial workload patterns. These concerns have been studied in the context of autoscaling cloud services and serverless platforms and are not unique to SLM-based inference. Production deployments typically incorporate rate limiting, admission control, and anomaly detection mechanisms to mitigate such risks. Additionally, SLMs offer inherent data privacy benefits by allowing localized, on device or private-cloud processing, significantly reducing the exposure associated with sending sensitive data to centralized LLM APIs.

Overall, these factors represent important considerations for production deployment but do not invalidate the core insights of the proposed elasticity model. Instead, they motivate future work focused on integrating system-level optimizations and empirical deployment studies using industry-standard orchestration frameworks.

Necessity of intelligent elastic scaling are as follows. The core research hypothesis, which concluded that the management of the static or naive resources is inefficient during the changing workloads of the AI, was proven by technical implementation. The simulated framework was able to model the dynamic scaling and predictive allocation.

The initial simulation using a *Random Policy* decisively validated the core research hypothesis: static or naive resource management strategies are inadequate for handling fluctuating AI inference workloads.

I. KEY FINDINGS FROM THE SIMULATION

The experimental results revealed several critical limitations of simplistic scaling approaches:

- The system exhibited a high percentage of volatility and a lack of correspondence between the ability to compute and demand.

- Random or fixed scaling either over-provisioned (unutilized GPUs) or under-provisioned (missed request).
- The existing dynamically loaded frequent request drops demonstrated the lack of reliability of the non-intelligent scaling.

These results support the need to have smart, intelligent scaling solutions to current AI inference tasks.

J. JUSTIFICATION FOR THE PREDICTIVE AND OPTIMIZATION FRAMEWORK

The experiment supports the necessity of the intelligent control system that will combine forecasting and decision making, including:

- Workload prediction models based on **hybrids of TCN and GRU**.
- **CNN-LSTM models** that are used to predict on a GPU.
- **Deep Reinforcement Learning (DRL)** optimizers like DQN or PPO in the control of real-time scaling [9], [17]. Predictive and control-based autoscaling approaches further demonstrate the effectiveness of integrating forecasting with adaptive decision-making in cloud-native environments [6]. Such integration will facilitate dynamic scaling that is proactive and efficient to workload dynamics.

K. FUTURE WORK AND IMPLICATIONS

Future studies will involve using DRL-based controllers into the predictive to realize real-time elasticity. This architecture offers a scalable, long-term route to the deployment of AI in the cloud by sustaining usage in ranges of 40–75% optimal operation, and achieving to 47% minor savings over any individual deployment of the stable, linear least usage, linear maximum deployment of the least-learned machine.

L. CONTRIBUTIONS

This work presents an exploratory framework to evaluate elastic SLM deployments, building on well-studied components such as autoscaling and small models. Our main contributions are:

- **Decision Framework for Elastic SLMs:** While previous work studies SLMs or scaling independently, we propose a framework to determine when elastic SLM deployments outperform static configurations, balancing model size, resource allocation, and workload requirements.
- **Cost–Quality–Latency Tradeoff Analysis:** While tradeoffs in NLP inference have been explored partially, we systematically analyze the interaction between computational cost, inference latency, and model quality under elastic scaling.
- **System-Level Perspective:** Although optimization and infrastructure issues are considered in isolation, we consider them together, focusing on how scaling strategies and model selection interact with performance and efficiency.

- **Simulation of Operational Metrics:** Although accuracy metrics are typically provided, we simulate latency, resource consumption, and cost with respect to workload to demonstrate deployment implications.
- **Preliminary Sustainability Considerations:** Although energy efficiency is typically studied in isolation, we offer a preliminary discussion of how the elastic deployment of smaller models could mitigate energy and resource waste during low-demand periods.

This research aims to investigate the tradeoffs between cost, latency, and utilization made possible by the elastic scaling of Small Language Models (SLMs) through analysis and experimentation. Although the findings are informative, there are some limitations to be considered.

M. SIMULATION-CENTRIC EVALUATION

The main evaluation in this research is simulation-centric and abstracts away from some real-world system dynamics such as the startup time of containers, network fluctuations, GPU overheads, and resource contention in multi-tenant environments. Although we have qualitatively talked about these aspects and justified our assumptions based on the systems literature, a full-fledged deployment would be needed to estimate the net effect.

N. LIMITED MODEL AND WORKLOAD SCOPE

Our analysis holds that the target workload can be supported by SLMs with acceptable quality. This may not be the case for complex reasoning tasks, long context queries, or highly specialized domains where large language models are still required. In such cases, hybrid models that direct complex queries to LLMs and high-volume routine queries to elastic SLMs would be more suitable. and high-volume routine requests to elastic SLMs may be more appropriate.

O. SIMPLIFIED LATENCY AND COST MODELS

Our latency model is based largely on the impact of utilization and queuing, while our cost model is based on normalized per-instance pricing assumptions. These models are intended to reflect general trends rather than exact costs or latency estimates for particular cloud vendors. In practice, actual systems may reflect non-linear pricing relationships, spot instance variation, or hardware differences.

P. LIMITED EMPIRICAL VALIDATION

Although we provide a minimal CPU-based validation experiment to verify the qualitative latency behavior, the evaluation does not include extensive cloud-scale experiments or traces from real-world production traffic. These experiments are left for future work and are required to comprehensively validate the scalability and robustness of the proposed framework.

These limitations are a result of the scoping decisions made to isolate and understand the underlying trade-offs of elastic inference-serving with SLMs, rather than a production-quality system.

V. CONCLUSION

In this work, we presented that despite significant advancements in natural language processing enabled by large language models, their static deployment remains inefficient due to high computational cost and latency. The research has demonstrated that when combined with efficient resource management, smaller language models have been capable of providing adequate performance due to moderate-complexity, high-volume workloads. Key findings are made using an analytical framework at the system level with autoscaling simulation like Kubernetes HPA and actual latency measurements of a CPU-based deployment of the DistilBERT. These findings have shown that elastic scaling has been successful in adjusting to the changing demand, maintaining target utilization, decreasing the cost of computation, and consistent performance with highly variable workloads. Moreover, it has been found to have considerable trade-offs between fixed and dynamic resource allocation. In general, elastic inference has been shown to be a viable and inexpensive approach to deploy language models in resource-constrained settings without substituting large models.

CONFLICTS OF INTEREST

The authors do not have any conflict of interest.

ACKNOWLEDGMENT

The authors sincerely thank Manipal Academy of Higher Education for the research support provided for this work and for covering the Article Processing Charges.

REFERENCES

- [1] T. Brown et al., "Language models are few-shot learners," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 33, 2020, pp. 1877–1901.
- [2] C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu, "Exploring the limits of transfer learning with a unified text-to-text transformer," *J. Mach. Learn. Res.*, vol. 21, no. 140, pp. 1–67, 2020.
- [3] P. Pipek, S. Canavan, S. Canavan, C. Capinha, J. M. W. Gippet, A. Novoa, P. Pyšek, A. T. Souza, S. Wang, and I. Jarić, "Sustainability of large language models: a user perspective," *Frontiers Ecol. Environ.*, vol. 23, no. 5, Jun. 2025, Art. no. e2856, doi: [10.1002/fee.2856](https://doi.org/10.1002/fee.2856).
- [4] M.-N. Tran, D.-D. Vu, and Y. Kim, "A survey of autoscaling in Kubernetes," in *Proc. 13th Int. Conf. Ubiquitous Future Netw. (ICUFN)*, Jul. 2022, pp. 263–265.
- [5] B. Jeong and Y.-S. Jeong, "Autoscaling techniques in cloud-native computing: A comprehensive survey," *Comput. Sci. Rev.*, vol. 58, Nov. 2025, Art. no. 100791.
- [6] S. Shim, A. Dhokariya, D. Doshi, S. Upadhye, V. Patwari, and J.-Y. Park, "Predictive auto-scaler for Kubernetes cloud," in *Proc. IEEE Int. Syst. Conf. (SysCon)*, Apr. 2023, pp. 1–6.
- [7] I. Pintye, J. Kovács, and R. Lovas, "Enhancing machine learning-based autoscaling for cloud resource orchestration," *J. Grid Comput.*, vol. 22, no. 4, pp. 1–23, Dec. 2024.
- [8] R. Moreno-Vozmediano, R. S. Montero, E. Huedo, and I. M. Llorente, "Efficient resource provisioning for elastic Cloud services based on machine learning techniques," *J. Cloud Comput.*, vol. 8, no. 1, pp. 1–18, 2019, doi: [10.1186/s13677-019-0128-9](https://doi.org/10.1186/s13677-019-0128-9).
- [9] C. Lekkala, "AI-driven dynamic resource allocation in cloud computing," *J. Artif. Intell. Mach. Learn. Data Sci.*, vol. 2, no. 2, pp. 450–456, 2024.
- [10] S. Gupta, "Scalability and resilience in distributed LLM training: A survey of modern techniques," *J. Int. Crisis Risk Commun. Res.*, vol. 8, no. S8, pp. 31–48, Aug. 2025.

- [11] Z. Gong, X. Gu, and J. Wilkes, "PRESS: PRedictive elastic resource scaling for cloud systems," in *Proc. Int. Conf. Netw. Service Manage.*, Oct. 2010, pp. 9–16.
- [12] C. Duman and S. Eken, "From reactive to predictive: A pattern-aware framework for kubernetes autoscaling with large language model integration," *J. Syst. Softw.*, vol. 237, Jul. 2026, Art. no. 112861.
- [13] Á. MacDermott, Q. Shi, M. Merabti, and K. Kifiyat, "Considering an elastic scaling model for cloud security," in *Proc. 8th Int. Conf. Internet Technol. Secured Trans. (ICITST-)*, Dec. 2013, pp. 150–155.
- [14] E. F. Coutinho, F. R. C. Sousa, P. A. L. Rego, D. G. Gomes, and J. N. D. Souza, "Elasticity in cloud computing: A survey," *Ann. Telecommun.*, vol. 70, nos. 7–8, pp. 289–309, 2014.
- [15] S. Chennupati, "Scalable cloud architectures for real-time AI: Dynamic resource allocation for inference optimization," *J. Comput. Sci. Technol. Stud.*, vol. 7, no. 3, pp. 690–700, May 2025.
- [16] M. A. K. Raiaan, M. S. H. Mukta, K. Fatema, N. M. Fahad, S. Sakib, M. M. J. Mim, J. Ahmad, M. E. Ali, and S. Azam, "A review on large language models: Architectures, applications, taxonomies, open issues and challenges," *IEEE Access*, vol. 12, pp. 26839–26874, 2024, doi: [10.1109/ACCESS.2024.3365742](https://doi.org/10.1109/ACCESS.2024.3365742).
- [17] A. Padhy, "DRL-KubeScale: A reinforcement learning based autoscaler for SLA-aware pod optimization in Kubernetes," in *Proc. 8th Int. Conf. Image Inf. Process. (ICIIP)*, Nov. 2025, pp. 748–753.
- [18] K. S. Hebbar, "Workload-aware machine learning for microservice scaling in Kubernetes," *Int. J. Comput. Experim. Sci. Eng.*, vol. 11, no. 4, pp. 9870–9880, Dec. 2025.
- [19] Q. Zhang, Z. Liu, and S. Pan, "The rise of small language models," *IEEE Intell. Syst.*, vol. 40, no. 1, pp. 30–37, Jan. 2025.
- [20] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models," *J. Mach. Learn. Res.*, vol. 23, no. 120, pp. 1–39, 2022.
- [21] M. Rizwan, M. F. Mushtaq, U. Akram, A. Mehmood, I. Ashraf, and B. Sahelices, "Depression classification from tweets using small deep transfer learning language models," *IEEE Access*, vol. 10, pp. 129176–129189, 2022.
- [22] Y. Shen, L. Heacock, J. Elias, K. D. Hentel, B. Reig, G. Shih, and L. Moy, "ChatGPT and other large language models are double-edged swords," *Radiology*, vol. 307, no. 2, Apr. 2023, Art. no. 230163.
- [23] R. Li, D. Fu, C. Shi, Z. Huang, and G. Lu, "Efficient LLMs training and inference: An introduction," *IEEE Access*, vol. 13, pp. 32944–32970, 2025.



SIMRAN AJIT KUMAR MOHANTY is currently pursuing the B.Tech. degree in computer science with Sir Padampat Singhanian University, Udaipur, India.

She was an Intern at ResAISHala Technologies, developing and deploying their homepage website integrating both front-end and back-end functionalities. She was also an Intern with the Defense Research and Development Organization (DRDO), Chandigarh, contributing to weather prediction models. Her research and project interests include full stack web development and AI-based environmental applications.



DEEPAK PARASHAR (Senior Member, IEEE) received the Ph.D. degree specializing in medical image analysis and AI and ML-based glaucoma detection. He is currently an Associate Professor with the School of Computer Engineering, Manipal Institute of Technology, Manipal Academy of Higher Education, Manipal, India. He has over a decade of experience in teaching and research. His research interests include artificial intelligence, machine learning, deep learning, computer vision, bio-medical image processing, and the IoT applications in healthcare. He has published numerous research papers and book chapters, guided several student projects, and he actively contributed as a Reviewer and a member of professional bodies, such as ACM and ISTE.



MITALI THAPA received the M.Tech. and Ph.D. degrees in computer engineering in 2012 and 2020, respectively. He is currently an Associate Professor with the Department of Computer Science and Engineering, Symbiosis Institute of Technology, Symbiosis International (Deemed University), Pune, Maharashtra, India. He has more than 20 years of academic experience. He has designed and conducted various faculty development programs, workshops, and national and international conferences as a Convener. He has several publications to his credit and has presented 15 research papers at international conferences organized by various central and state universities and government-affiliated engineering colleges. He is currently guiding six Ph.D. scholars. His research interests include routing protocols, security, blockchain, and image processing. He serves as a reviewer for various internationally reputed journals.



KAMAL KANT HIRAN (Senior Member, IEEE) has over 21 years of rich experience in academia and research in Asia, Africa, and Europe. He is recognized among the world's Top 2% Scientists by the Stanford-Elsevier List 2025 for his outstanding research contributions. He has published 125 scientific peer-reviewed research articles in leading journals, 40 international books with reputable publishers, 11 Indian patents, and two Australian patents. He is currently the Founder of the IEEE Liberia Subsection Chair, Liberia, West Africa.



ANKUR GOYAL received the M.Tech. degree in blockchain, image processing, and computer engineering from Rajasthan Technical University, Kota, India, in 2012, and the Ph.D. degree, in 2020.

He is currently an Associate Professor with the Department of Engineering and Technology, Symbiosis International (Deemed University), Symbiosis Institute of Technology, Pune, Maharashtra, India. He has over 20 years of academic experience. He has designed and conducted various faculty development programs, workshops, and national and international conferences as a convener. He has several publications to his credit and has presented 15 research papers at international conferences organized by various central and state universities and government affiliated engineering colleges. He is also guiding six Ph.D. scholars.

...